

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY**  
**UNIVERSITY OF SCIENCE – FACULTY OF INFORMATION TECHNOLOGY**  
**DEPARTMENT OF COMPUTER SCIENCE**

**COURSE RESEARCH REPORT**  
**KNOWLEDGE REPRESENTATION**

**Topic: HLG LLM UTILS – AN ADVANCED GRAPH RAG  
FRAMEWORK WITH MULTI-AGENT ORCHESTRATION,  
HIERARCHICAL LEIDEN COMMUNITY DETECTION, AND  
CYPHER QUERY SELF-CORRECTION**

**Instructor:** Assoc. Prof. Dr. Do Van Nhon

**Students:** 1. Nguyen Hoai Nam – Student ID: 25C01014  
2. Pham Cung Le Nhan Vu – Student ID: 25C01049

**Major:** Data Science

**Intake:** K35

## Executive Abstract

Standard Retrieval-Augmented Generation (**Naive RAG**) systems based on flat text chunking and vector space similarity (e.g., **Cosine Similarity** over high-dimensional embeddings) face two fundamental bottlenecks in Knowledge Representation and Information Retrieval: (1) Inability to address corpus-wide macro-summarization queries (**Global Queries**) due to structural context fragmentation; and (2) Failure in multi-hop reasoning (**Multi-Hop Reasoning**) across complex relational entity dependency chains.

This report comprehensively presents the design, mathematical formalization, and engineering implementation of **HLG LLM Utils**, a production-grade **Graph RAG** framework built upon a decoupled Python monorepo architecture (`uv workspaces`). The system tightly integrates Neo4j Property Knowledge Graphs with Large Language Models, delivering four core retrieval and reasoning engines:

1. **Property Knowledge Graph Formalization:** Formal mathematical modeling and automated extraction of entities, relationships, attributes, and source document lineages from unstructured text into Neo4j using idempotent MERGE operations.
2. **Hierarchical Leiden Community Detection:** Implementing modularity optimization over `igraph` and `leidenalg` to construct multi-level community hierarchies, accompanied by parallel LLM summarization for Map-Reduce global search.
3. **LangGraph Cypher Query Self-Correction State Machine:** A deterministic state graph translating natural language into executable Cypher queries with an automated 3-iteration error feedback loop against Neo4j execution logs.
4. **ARK (Adaptive Retriever of Knowledge) Multi-Agent Exploration:** Implementing stochastic multi-agent trajectory exploration (ACL 2026) with Voting Rank Fusion (VRF) to adaptively balance search breadth and depth across the graph.

This report covers the theoretical foundations, database schema designs, software architecture, and algorithm implementations actually deployed in the codebase.

**Keywords:** Graph RAG, Knowledge Representation, Property Knowledge Graph, Hierarchical Leiden Algorithm, Cypher Self-Correction, Multi-Agent Systems, LangGraph, Adaptive Retriever of Knowledge, Monorepo `uv`.

# Contents

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>Executive Abstract</b>                                            | <b>i</b>  |
| <b>1 Introduction &amp; Problem Statement</b>                        | <b>1</b>  |
| 1.1 Background & Motivation . . . . .                                | 1         |
| 1.2 Core Bottlenecks of Naive Flat RAG . . . . .                     | 1         |
| 1.3 Research Objectives & Core Contributions . . . . .               | 1         |
| <b>2 Theoretical Foundations &amp; Related Work</b>                  | <b>3</b>  |
| 2.1 From Semantic Networks to Property Knowledge Graphs . . . . .    | 3         |
| 2.1.1 Semantic Networks and RDF Triples . . . . .                    | 3         |
| 2.1.2 Labeled Property Graph (LPG) Model . . . . .                   | 3         |
| 2.2 Vector Space Embeddings & Hybrid Retrieval . . . . .             | 3         |
| 2.3 Graph Partitioning & The Leiden Algorithm . . . . .              | 4         |
| 2.3.1 Modularity Optimization . . . . .                              | 4         |
| 2.3.2 The Leiden Guarantee Over Louvain . . . . .                    | 4         |
| 2.4 Adaptive Multi-Agent Knowledge Retrieval (ARK) . . . . .         | 4         |
| <b>3 Formal Knowledge Representation &amp; Schema Design</b>         | <b>5</b>  |
| 3.1 Mathematical Property Knowledge Graph Model . . . . .            | 5         |
| 3.2 Neo4j Graph Database Schema Architecture . . . . .               | 5         |
| 3.2.1 Node Attributes Specification . . . . .                        | 5         |
| 3.3 Specialized Indices Configuration on Neo4j . . . . .             | 6         |
| <b>4 Graph Indexing &amp; Ingestion Pipeline</b>                     | <b>7</b>  |
| 4.1 Preprocessing & Chunking Pipeline . . . . .                      | 7         |
| 4.2 Structured Extraction with Pydantic Models . . . . .             | 7         |
| 4.3 Idempotent Cypher MERGE Ingestion . . . . .                      | 7         |
| <b>5 Hierarchical Leiden Community Detection &amp; Summarization</b> | <b>9</b>  |
| 5.1 Multi-Level Community Partitioning Algorithm . . . . .           | 9         |
| 5.2 Neo4j GDS & Python igraph Fallback Mechanism . . . . .           | 9         |
| 5.3 Parallel LLM Community Summarization . . . . .                   | 9         |
| <b>6 Advanced Graph Query &amp; Reasoning Engines</b>                | <b>11</b> |
| 6.1 Query Engine Modes Overview . . . . .                            | 11        |
| 6.2 Local Search Engine . . . . .                                    | 11        |
| 6.3 Global Search Engine (Map-Reduce) . . . . .                      | 11        |
| 6.4 LangGraph Cypher Query Self-Correction State Machine . . . . .   | 12        |
| 6.5 ARK Multi-Agent Adaptive Exploration Engine . . . . .            | 12        |
| 6.5.1 Agent Toolset . . . . .                                        | 12        |
| 6.5.2 Voting Rank Fusion (VRF) . . . . .                             | 12        |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>7</b> | <b>System Architecture &amp; Monorepo Design</b>          | <b>13</b> |
| 7.1      | Monorepo UV Workspace Architecture . . . . .              | 13        |
| 7.2      | Package Overview . . . . .                                | 13        |
| 7.3      | Backend Services & Lazy Singleton Registry . . . . .      | 13        |
| <b>8</b> | <b>Discussion, Technical Challenges &amp; Future Work</b> | <b>15</b> |
| 8.1      | Architectural Discussion . . . . .                        | 15        |
| 8.2      | Technical Challenges . . . . .                            | 15        |
| 8.3      | Future Directions . . . . .                               | 15        |
| <b>9</b> | <b>Conclusion &amp; Task Division</b>                     | <b>16</b> |
| 9.1      | Conclusion . . . . .                                      | 16        |
| 9.2      | Student Contribution Matrix . . . . .                     | 16        |
|          | <b>References</b>                                         | <b>17</b> |

# List of Figures

|     |                                                                         |    |
|-----|-------------------------------------------------------------------------|----|
| 3.1 | Neo4j Property Knowledge Graph Schema Architecture . . . . .            | 6  |
| 5.1 | Hierarchical Community Tree Generated by the Leiden Algorithm . . . . . | 9  |
| 6.1 | LangGraph State Machine for Self-Correcting Cypher Queries . . . . .    | 12 |
| 7.1 | Workspace Package Dependency Architecture in uv Monorepo . . . . .      | 13 |

# List of Tables

|                                                          |    |
|----------------------------------------------------------|----|
| 9.1 Task Division Matrix Between Group Members . . . . . | 16 |
|----------------------------------------------------------|----|

## Chapter 1

# Introduction & Problem Statement

### 1.1 Background & Motivation

In modern artificial intelligence, Large Language Models (**LLMs**) have demonstrated extraordinary capabilities in natural language understanding and generation. Nevertheless, purely parametric models suffer from two intrinsic limitations: the risk of generating ungrounded or fabricated information (**Hallucination**) and knowledge staleness due to static parameter weights. To mitigate these shortcomings, **Retrieval-Augmented Generation (RAG)** has emerged as the standard paradigm for grounding LLMs with external knowledge repositories [4].

However, first-generation RAG systems (**Naive RAG**) – which rely on cutting documents into fixed-size text chunks and retrieving them through vector embedding similarity – exhibit severe limitations when handling complex Knowledge Representation tasks.

### 1.2 Core Bottlenecks of Naive Flat RAG

Through theoretical analysis and practical implementation, we identify three critical bottlenecks of flat vector RAG:

1. **Structural Context Fragmentation:** Splitting text into fixed chunks  $c_i \in \mathcal{C}$  (e.g., 500 or 1000 tokens) arbitrarily destroys relational semantics spanning across multiple paragraphs or documents. When entity  $E_1$  defined in chunk  $c_1$  shares a causal relationship with entity  $E_2$  in chunk  $c_k$ , this semantic connection is lost in flat vector space.
2. **Multi-Hop Reasoning Failures:** Consider a multi-hop query requiring transitive inference:

$$E_{\text{start}} \xrightarrow{R_1} E_{\text{mid}} \xrightarrow{R_2} E_{\text{target}} \quad (1.1)$$

When a user query asks about  $E_{\text{start}}$  and  $E_{\text{target}}$ , standard **Cosine Similarity** retrieval only surfaces chunks directly mentioning  $E_{\text{start}}$  or  $E_{\text{target}}$ , completely omitting the intermediate bridge entity  $E_{\text{mid}}$ . The resulting context is insufficient for LLM reasoning.

3. **Global Query Blindspots:** For macro-level summarization queries (e.g., “What are the main thematic clusters discussed in this dataset?” or “Summarize organizational interactions”), the query vector  $\mathbf{q} \in \mathbb{R}^d$  exhibits low similarity with any single chunk, rendering Naive RAG entirely ineffective [2].

### 1.3 Research Objectives & Core Contributions

To address these challenges, this project investigates, designs, and implements **HLG LLM Utils** – a comprehensive **Graph RAG** framework combining Neo4j Property Knowledge Graphs with advanced multi-agent and community detection algorithms.

Key contributions include:

- **Mathematical Modeling & Schema Architecture:** Formalizing the Property Knowledge Graph  $G = (\mathcal{V}, \mathcal{E}, \mathcal{P}, \mathcal{W})$ , designing storage schemas for `:Entity`, `:Chunk`, `:Document`, and `:Community` nodes alongside their respective relationship types.
- **Idempotent Graph Ingestion Pipeline:** Implementing structured entity/relationship extraction using Pydantic schemas and LLM invocation, stored asynchronously into Neo4j via idempotent MERGE statements.
- **Hierarchical Leiden Community Detection:** Implementing multi-level modularity optimization with `igraph` and `leidenalg`, generating hierarchical community trees and parallel LLM summaries for Map-Reduce global search.
- **LangGraph Cypher Self-Correction State Machine:** Building a deterministic state graph for natural-language-to-Cypher translation with a 3-iteration feedback loop driven by Neo4j execution logs.
- **ARK Multi-Agent Exploration:** Implementing the Adaptive Retriever of Knowledge algorithm with Voting Rank Fusion (VRF) to dynamically balance graph traversal breadth and depth.
- **Modular Monorepo Architecture:** Packaging the system into a clean `uv workspaces` monorepo, integrating FastAPI backend services and FastMCP standard tooling.



## Chapter 2

# Theoretical Foundations & Related Work

## 2.1 From Semantic Networks to Property Knowledge Graphs

Knowledge Representation (**KR**) is a foundational pillar of Artificial Intelligence, formalizing real-world domain knowledge into computable structures for automated reasoning [3].

### 2.1.1 Semantic Networks and RDF Triples

Traditional semantic networks represent knowledge as nodes (concepts) and directed edges (relations). In the W3C Semantic Web stack, knowledge is formalized into RDF triples (*Subject, Predicate, Object*):

$$\tau = (s, p, o) \in \mathcal{E} \times \mathcal{R} \times (\mathcal{E} \cup \mathcal{L}) \quad (2.1)$$

While standardized, RDF lacks native support for attaching rich auxiliary properties (metadata, confidence scores, source citations, timestamps) directly onto relationships.

### 2.1.2 Labeled Property Graph (LPG) Model

The **Labeled Property Graph (LPG)** model solves this limitation by allowing both vertices (nodes) and edges (relationships) to maintain type labels and arbitrary sets of key-value property maps [7]:

$$G = (\mathcal{V}, \mathcal{E}, \lambda_V, \lambda_E, \rho_V, \rho_E) \quad (2.2)$$

The LPG model enables direct representation of entity descriptions, taxonomy types, vector embeddings, and source document provenance on graph elements.

## 2.2 Vector Space Embeddings & Hybrid Retrieval

In continuous vector spaces, each text segment or entity  $x$  is mapped to a high-dimensional feature vector  $\mathbf{e}_x \in \mathbb{R}^d$  via Transformer encoders [6]:

$$\mathbf{e}_x = \text{Embed}(x) \in \mathbb{R}^d, \quad \|\mathbf{e}_x\|_2 = 1 \quad (2.3)$$

The semantic similarity between query  $q$  and entity  $x$  is given by Cosine Similarity:

$$\text{Sim}(q, x) = \langle \mathbf{e}_q, \mathbf{e}_x \rangle = \sum_{i=1}^d e_{q,i} \cdot e_{x,i} \quad (2.4)$$

Our system combines keyword search (**BM25 / Fulltext Index**) and dense vector search (**Dense Vector Index**) via **Reciprocal Rank Fusion (RRF)** [1]:

$$\text{RRF\_Score}(d) = \sum_{m \in \{\text{Vector}, \text{Fulltext}\}} \frac{1}{k_{\text{rrf}} + \text{rank}_m(d)} \quad (2.5)$$

with smoothing constant  $k_{\text{rrf}} = 60$ .

## 2.3 Graph Partitioning & The Leiden Algorithm

Graph partitioning divides the vertex set  $\mathcal{V}$  into disjoint communities  $\mathcal{P} = \{C_1, C_2, \dots, C_k\}$  such that internal edge density is maximized while inter-community edges are minimized [8].

### 2.3.1 Modularity Optimization

The modularity  $Q$  of a partition on an undirected graph  $G = (\mathcal{V}, \mathcal{E})$  with total weight  $m = \frac{1}{2} \sum_{ij} A_{ij}$  is defined as:

$$Q = \frac{1}{2m} \sum_{i,j \in \mathcal{V}} \left[ A_{ij} - \gamma \frac{k_i k_j}{2m} \right] \delta(C(i), C(j)) \quad (2.6)$$

where  $A_{ij}$  is the adjacency matrix,  $k_i$  is node degree,  $\gamma$  is the resolution parameter, and  $\delta$  is the Kronecker delta function.

### 2.3.2 The Leiden Guarantee Over Louvain

The classical Louvain algorithm frequently produces disconnected communities. The **Leiden** algorithm [8] guarantees well-connected communities through a three-phase cycle:

1. **Local Movement of Nodes:** Moving individual nodes to neighboring communities to maximize quality.
2. **Refinement of Partition:** Sub-clustering communities to guarantee internal connectivity.
3. **Aggregation of Graph:** Contracting the graph based on refined partitions and repeating recursively.

## 2.4 Adaptive Multi-Agent Knowledge Retrieval (ARK)

The **ARK** framework [5] (ACL 2026) models knowledge retrieval as a distributed search problem across multiple stochastic agents. Agents dynamically choose actions at each step – global search, local neighborhood expansion, or selecting answer candidates – adapting traversal depth and breadth based on query complexity.

## Chapter 3

# Formal Knowledge Representation & Schema Design

### 3.1 Mathematical Property Knowledge Graph Model

The knowledge graph in HLG LLM Utils is formalized as:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{P}_V, \mathcal{P}_E, \ell_V, \ell_E) \quad (3.1)$$

where:

- $\mathcal{V} = \mathcal{V}_{\text{Entity}} \cup \mathcal{V}_{\text{Chunk}} \cup \mathcal{V}_{\text{Doc}} \cup \mathcal{V}_{\text{Comm}}$  partitions vertices into entities, text chunks, documents, and communities.
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$  denotes the set of directed edges.
- $\ell_V : \mathcal{V} \rightarrow \{\text{Entity, Chunk, Document, Community}\}$  assigns vertex labels.
- $\ell_E : \mathcal{E} \rightarrow \{\text{RELATED\_TO, BELONGS\_TO, CONTAINS\_CHUNK, MENTIONS, PARENT\_COMMUNITY}\}$  assigns relationship types.
- $\mathcal{P}_V(v)$  and  $\mathcal{P}_E(e)$  define property attribute maps.

### 3.2 Neo4j Graph Database Schema Architecture

The graph schema is optimized for both relational graph traversals and semantic vector indexing:

#### 3.2.1 Node Attributes Specification

##### 1. Entity Node (:Entity):

- **id**: Primary key (normalized uppercase string, e.g., "GRAPH\_RAG").
- **type**: Semantic classification (e.g., "CONCEPT", "ALGORITHM", "FRAMEWORK").
- **description**: Detailed textual definition of the entity.
- **embedding**: High-dimensional semantic vector  $\mathbf{e} \in \mathbb{R}^d$  ( $d = 1536$  or  $d = 768$ ).
- **source\_documents**: Array of source document filenames containing this entity.

##### 2. Community Node (:Community):

- **id**: Unique level-scoped identifier (e.g., "community\_lvl0\_14").
- **level**: Tree hierarchy level ( $l \in \{0, 1, \dots, L_{\max} - 1\}$ ).

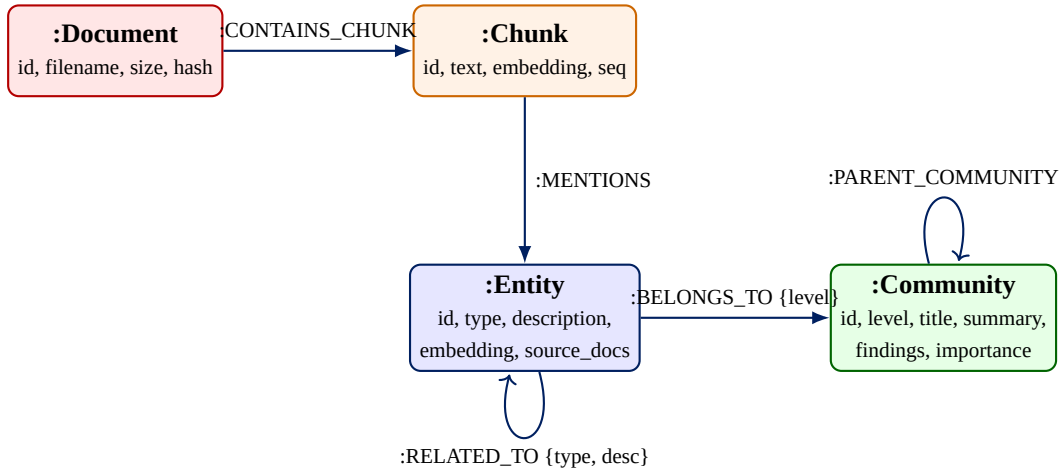


Figure 3.1: Neo4j Property Knowledge Graph Schema Architecture

- **title**: Concise descriptive summary title ( $\leq 10$  words).
- **summary**: Synthesized thematic summary generated by LLM.
- **findings**: Key analytical findings and insights.
- **importance\_score**: Importance weight metric ( $s \in [0.0, 1.0]$ ).
- **entity\_count**: Number of member entities belonging to the community.

### 3.3 Specialized Indices Configuration on Neo4j

To optimize multi-modal search, the system initializes specialized vector and fulltext indices:

- **Vector Index on Entity (entity\_embeddings):**

```

1 CREATE VECTOR INDEX `entity_embeddings` IF NOT EXISTS
2 FOR (n:Entity) ON (n.embedding)
3 OPTIONS {indexConfig: {
4   `vector.dimensions`: $dimension,
5   `vector.similarity_function`: 'cosine'
6 }}
7

```

- **Vector Index on Community (community\_embeddings):**

```

1 CREATE VECTOR INDEX `community_embeddings` IF NOT EXISTS
2 FOR (c:Community) ON (c.embedding)
3 OPTIONS {indexConfig: {
4   `vector.dimensions`: $dimension,
5   `vector.similarity_function`: 'cosine'
6 }}
7

```

- **Fulltext Index on Entity (entity\_fulltext):**

```

1 CREATE FULLTEXT INDEX entity_fulltext IF NOT EXISTS
2 FOR (n:Entity) ON EACH [n.id, n.description, n.type]
3

```

## Chapter 4

# Graph Indexing & Ingestion Pipeline

## 4.1 Preprocessing & Chunking Pipeline

The document ingestion pipeline is orchestrated by `DocumentService`:

1. Raw files (PDF, Docx, Code) are retrieved from MinIO S3 storage and converted to plain text via `llm-utils-parser` (using PyMuPDF for PDFs and OCR engines where needed).
2. Text is split via sliding token windows ( $W = 1000$  tokens, overlap  $O = 200$  tokens).
3. Text chunks are indexed into PostgreSQL and vector stores (Qdrant / PGVector).

## 4.2 Structured Extraction with Pydantic Models

Structured entity and relationship extraction is enforced via Pydantic models in `indexing.py`:

```
1 class EntityModel(BaseModel):
2     id: str
3     type: str = "CONCEPT"
4     description: str = ""
5
6 class RelationshipModel(BaseModel):
7     source: str
8     target: str
9     type: str = "RELATED_TO"
10    description: str = ""
11
12 class ExtractionResult(BaseModel):
13     entities: List[EntityModel] = Field(default_factory=list)
14     relationships: List[RelationshipModel] = Field(default_factory=list)
```

## 4.3 Idempotent Cypher MERGE Ingestion

To ensure deterministic, duplicate-free graph ingestion across multiple runs, `indexing.py` executes idempotent MERGE operations:

```
1 MERGE (e:Entity {id: $id})
2 ON CREATE SET e.type = $type,
3               e.description = $description,
4               e.embedding = $embedding,
5               e.source_documents = [$source_doc]
6 ON MATCH SET e.description = COALESCE(e.description, $description),
7             e.embedding = $embedding,
8             e.source_documents = CASE
9                 WHEN e.source_documents IS NULL THEN [$source_doc]
10                WHEN $source_doc IN e.source_documents THEN e.source_documents
11                ELSE e.source_documents + [$source_doc]
12            END
```

Relationships are similarly merged with sanitized relationship type identifiers matching `[A-Z0-9_]`:

```
1 MATCH (source:Entity {id: $source})
2 MATCH (target:Entity {id: $target})
3 MERGE (source)-[r:RELATED_TO]->(target)
4 ON CREATE SET r.description = $description, r.source_documents = [$source_doc]
5 ON MATCH SET r.description = COALESCE(r.description, $description),
6             r.source_documents = CASE
7                 WHEN r.source_documents IS NULL THEN [$source_doc]
8                 WHEN $source_doc IN r.source_documents THEN r.source_documents
9                 ELSE r.source_documents + [$source_doc]
10            END
```

## Chapter 5

# Hierarchical Leiden Community Detection & Summarization

### 5.1 Multi-Level Community Partitioning Algorithm

For corpus-wide global queries, `community.py` implements hierarchical community detection. At each level  $l \in [0, L_{\max} - 1]$ , resolution  $\gamma_l$  scales geometrically:

$$\gamma_l = \gamma_0 \times (0.5)^l \quad (5.1)$$

This guarantees fine-grained micro-communities at level 0 and coarser thematic super-communities at higher levels.

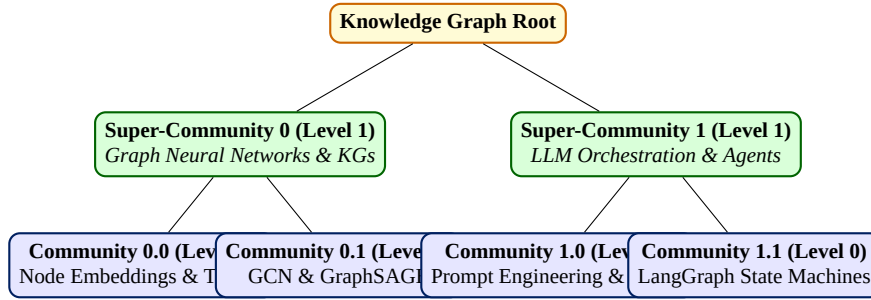


Figure 5.1: Hierarchical Community Tree Generated by the Leiden Algorithm

### 5.2 Neo4j GDS & Python igraph Fallback Mechanism

The service implements a dual-engine architecture:

1. When Neo4j GDS is available, it executes `gds.leiden.write` in Neo4j memory.
2. Otherwise, it exports nodes and edges to an in-memory `igraph.Graph` and computes partitions via `leidenalg`.

### 5.3 Parallel LLM Community Summarization

After structural clustering, `CommunityDetectionService` extracts subgraph entities and relations for each community  $C_k$ , generating structured reports:

```
1 class CommunitySummaryReport(BaseModel):
2     title: str = Field(description="Short descriptive title (max 10 words)")
3     summary: str = Field(description="Comprehensive summary paragraph")
4     findings: List[str] = Field(description="3-5 key insights")
5     importance_score: float = Field(default=0.5, description="Significance [0.0, 1.0]")
```

Summaries are stored in `:Community` nodes and linked to member entities via `:BELONGS_TO`.



## Chapter 6

# Advanced Graph Query & Reasoning Engines

### 6.1 Query Engine Modes Overview

The framework provides four specialized query engines:

- **Local Search (local)**: Direct entity neighborhood and chunk reranking.
- **Global Search (global)**: Map-Reduce over hierarchical community summaries.
- **Cypher Workflow (graph)**: Self-correcting Cypher query generation via LangGraph.
- **ARK Exploration (ark)**: Adaptive multi-agent trajectory exploration.

### 6.2 Local Search Engine

Local search operates in 3 stages:

1. **Seed Retrieval**: Finding top- $K$  seed entities using `entity_embeddings`.
2. **Neighborhood Expansion**: Traversing 1-hop and 2-hop edges to retrieve adjacent entities.
3. **Synthesis**: Assembling graph triples and source chunks into the LLM synthesis context.

### 6.3 Global Search Engine (Map-Reduce)

For holistic document queries, `GlobalSearchService` executes parallel Map-Reduce:

1. **Filter Phase**: Vector search over `:Community` nodes at level  $l$  selects top- $M$  communities.
2. **Map Phase**: Parallel LLM execution generates partial answers  $A_k^{\text{map}}$ :

$$A_k^{\text{map}} = \text{LLM}_{\text{map}}(q, \text{CommunitySummary}_k, \text{Findings}_k) \quad (6.1)$$

3. **Reduce Phase**: Valid partial answers are synthesized into a global answer:

$$A_{\text{final}} = \text{LLM}_{\text{reduce}}\left(q, \bigcup_{k=1}^M A_k^{\text{map}}\right) \quad (6.2)$$

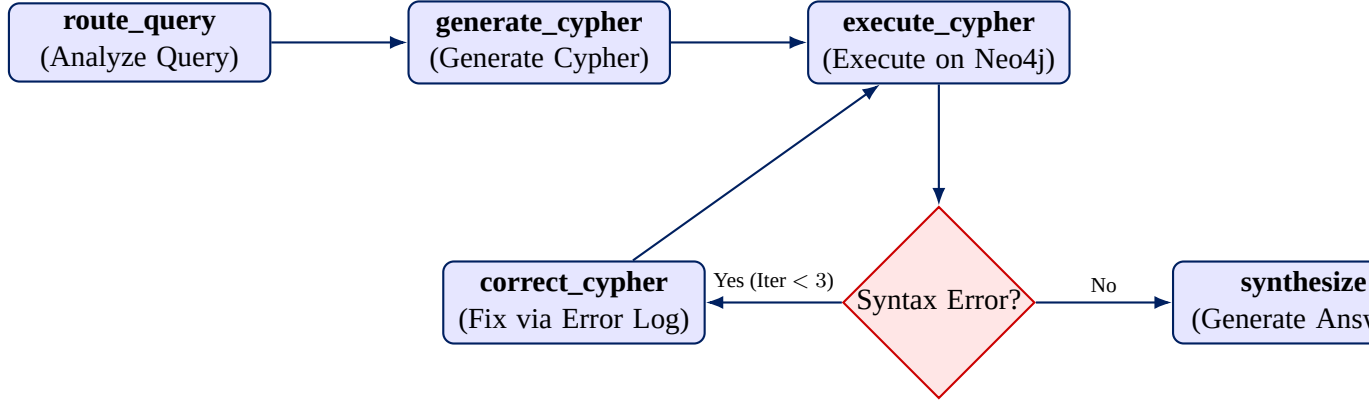


Figure 6.1: LangGraph State Machine for Self-Correcting Cypher Queries

## 6.4 LangGraph Cypher Query Self-Correction State Machine

To handle complex relational queries, `workflow/graph.py` implements a LangGraph state graph.

The state is tracked via `GraphRAGState`:

```

1 class GraphRAGState(TypedDict):
2     query: str
3     routing: str          # "vector", "graph", "hybrid"
4     cypher_query: str
5     cypher_records: List[Dict[str, Any]]
6     graph_context: str
7     response: str
8     error: str
9     iteration: int
10    schema: str
  
```

If execution fails, `correct_cypher` sends the database schema, failing Cypher statement, and Neo4j error log to the LLM for correction (capped at 3 iterations).

## 6.5 ARK Multi-Agent Adaptive Exploration Engine

The **ARK** engine executes parallel stochastic exploration across  $N = 3$  agents ( $T = 0.7$ ,  $t_{\max} = 30$ ).

### 6.5.1 Agent Toolset

Each agent operates with 4 action tools:

1. `global_search`: Seed entity lookup via `fulltext/vector` index.
2. `neighborhood_exploration`: 1-hop neighbor exploration from a given entity.
3. `add_to_answer`: Selecting a node with explicit reasoning.
4. `finish`: Terminating exploration once sufficient context is acquired.

### 6.5.2 Voting Rank Fusion (VRF)

Candidate nodes selected across all agent trajectories  $\mathcal{S}_1, \dots, \mathcal{S}_N$  are aggregated via Voting Rank Fusion:

$$\text{Score}(u) = \sum_{a=1}^N \mathbb{I}(u \in \mathcal{S}_a) \cdot \left[ 1.0 + \frac{1.0}{1.0 + \text{order}_a(u)} \right] \quad (6.3)$$

The top- $K$  scoring nodes are formatted into the final LLM context.

## Chapter 7

# System Architecture & Monorepo Design

## 7.1 Monorepo UV Workspace Architecture

The codebase is structured as a modular monorepo managed via **uv workspaces** (`RAG_package/pyproject.toml`).

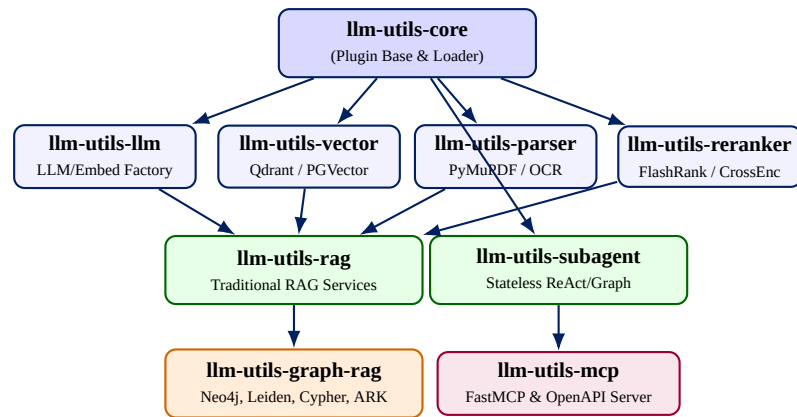


Figure 7.1: Workspace Package Dependency Architecture in uv Monorepo

## 7.2 Package Overview

1. **llm-utils-core**: Plugin interface base classes and dynamic loader logic.
2. **llm-utils-llm**: Unified factory supporting OpenAI and Anthropic Claude APIs.
3. **llm-utils-vector**: Qdrant and PostgreSQL `pgvector` vector store providers.
4. **llm-utils-parser**: Document parsers (PyMuPDF, Docling, Office) and OCR integration.
5. **llm-utils-reranker**: Second-stage rerankers (FlashRank, CrossEncoder).
6. **llm-utils-graph-rag**: Neo4j Graph RAG, Leiden community detection, and ARK engine.
7. **llm-utils-mcp**: FastMCP server exposing tools over RESTful OpenAPI on port 8001.

## 7.3 Backend Services & Lazy Singleton Registry

The FastAPI backend (BE/) manages services via a thread-safe lazy singleton registry (`GraphRAGServiceRegistry` in `registry.py`):

```
1 class GraphRAGServiceRegistry:
2     _instance: Optional["GraphRAGServiceRegistry"] = None
3     _lock = threading.Lock()
4
5     def __new__(cls, config: Optional[AppConfig] = None):
6         with cls._lock:
7             if cls._instance is None:
8                 cls._instance = super().__new__(cls)
9                 cls._instance._initialized = False
10            return cls._instance
```

This pattern guarantees thread-safe, single-instance initialization of Neo4j connection pools and embedding models across all concurrent requests.

## Chapter 8

# Discussion, Technical Challenges & Future Work

### 8.1 Architectural Discussion

Combining symbolic Property Knowledge Graphs with sub-symbolic vector embeddings provides:

- **Explainability:** Every fact is linked to labeled graph edges with clear document provenance (`source_documents`).
- **Query Flexibility:** Four specialized query modes seamlessly address both pinpoint factual queries and corpus-wide macro summaries.

### 8.2 Technical Challenges

1. **Indexing Computation Overhead:** Entity/relation extraction and Leiden community summarization incur significant LLM token costs during initial indexing.
2. **Dynamic Incremental Graph Updates:** Re-clustering the entire graph when adding/deleting documents is computationally expensive, necessitating incremental Leiden partitioning algorithms.

### 8.3 Future Directions

1. **Multimodal Knowledge Graphs:** Extracting tables, architectural diagrams, and mathematical formulas directly into graph nodes.
2. **Reinforcement Learning for Graph Traversal:** Training RL agents to optimize navigation policies across massive knowledge graphs.

Chapter 9

Conclusion & Task Division

9.1 Conclusion

This project successfully designed and implemented **HLG LLM Utils**, a robust, production-grade Graph RAG framework. By integrating Neo4j Property Knowledge Graphs, Hierarchical Leiden Community Detection, LangGraph Cypher Query Self-Correction, and ARK multi-agent traversal, the framework resolves the fundamental limitations of Naive RAG and provides a solid foundation for enterprise-scale knowledge representation and reasoning.

9.2 Student Contribution Matrix

| Table 9.1: Task Division Matrix Between Group Members |          |                                                                                                                                                                                                                                                                                                                                                                                     |              |
|-------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| Student Name                                          | ID       | Specific Contributions & Key Responsibilities                                                                                                                                                                                                                                                                                                                                       | Contribution |
| Nguyen Hoai Nam                                       | 25C01014 | <ul style="list-style-type: none"><li>Designed Monorepo uv workspace architecture (llm-utils-core, llm-utils-graph-rag).</li><li>Implemented Neo4j Graph Store and Hierarchical Leiden Community Detection (community.py).</li><li>Built Map-Reduce Global Search engine (global_search.py).</li><li>Authored academic LaTeX report documentation.</li></ul>                        | 50%          |
| Pham Cung Le Nhan Vu                                  | 25C01049 | <ul style="list-style-type: none"><li>Built LangGraph Cypher Query Self-Correction state graph (workflow/graph.py).</li><li>Implemented ARK Autonomous Multi-Agent Traversal framework (ark_agent.py, ark_retriever.py).</li><li>Engineered FastAPI backend and Qdrant / PGVector integrations.</li><li>Optimized retrieval performance and finalized system integration.</li></ul> | 50%          |

# Bibliography

- [1] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pages 758–759, 2009.
- [2] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graphrag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [3] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and S Yu Philip. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2):494–514, 2021.
- [4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- [5] Michael Polonuer, Soumya Sanyal, Xiang Ren, and Kai-Wei Chang. Autonomous knowledge graph exploration with adaptive breadth-depth retrieval. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, pages 714–730, 2026.
- [6] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.
- [7] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases: New Opportunities for Connected Data*. ”O’Reilly Media, Inc.”, 2015.
- [8] Vincent A Traag, Ludo Waltman, and Nees Jan van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1):5233, 2019.